Week 6

From Muscles to Neurons

What Can Motor Cortex Tell Us
That Muscles Cannot?

Section	Question it answers
1. Going Upstream	What generates the muscle patterns we have been classifying?
2. Cosine Tuning	How do motor cortex neurons encode reach direction?
3. Modeling Spike Counts	Why do we need the Poisson GLM for neural data?
4. Encoding	What is each neuron doing during the reach?
5. Decoding	Can we recover reach direction from neural activity?
6. The Clinical Question	Do neurons carry more diagnostic information than muscles?
7. Bringing It Together	Representation vs algorithm — which matters more?

1. Going Upstream

In Week 5, we discovered that raw EMG — six peak muscle amplitudes — can distinguish healthy from impaired subjects with 85% accuracy and an AUC of 0.94. The L1 regularization path told us *which* muscles carry the diagnostic signal: the proximal extensors at the shoulder and elbow.

But muscles do not act on their own. Every pattern of muscle activation during a reach originates from neural commands in **primary motor cortex (M1)**. The motor pathway flows from cortex to spinal cord to muscle to movement:

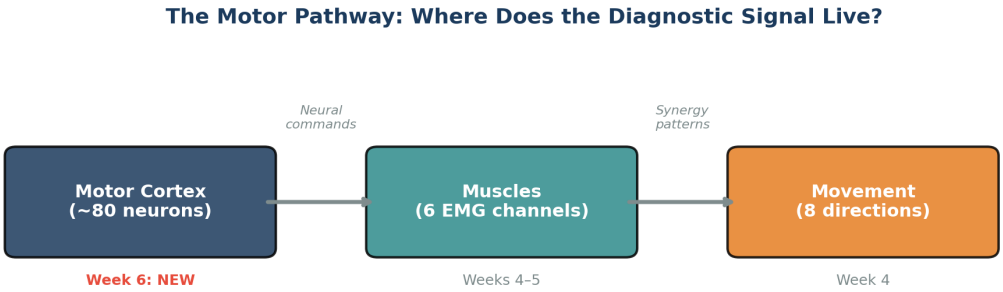

Figure 1. The motor pathway. Motor cortex neurons generate the commands that drive muscle activation patterns (synergies), which in turn produce movement. Weeks 4–5 worked with the muscle and movement layers. This week, we add the neural layer.

This opens a natural question: if the impairment shows up in muscle synergies (Week 4) and in classification performance (Week 5), does it also leave a signature in the neural activity? And which layer of the pathway carries *more* diagnostic information — the muscles, or the neurons that drive them?

A practical note: in a real clinical setting, we would not have access to single-neuron recordings from motor cortex — that requires invasive electrodes that are reserved for research or brain-computer interface applications. But our purpose here is not to build a deployable diagnostic tool. It is to use machine learning as a **lens** for understanding the neurobiology itself: by asking a classifier to distinguish healthy from impaired at each level of the motor pathway, we learn where the impairment signature is strongest, how it transforms as signals flow from cortex to muscle, and what is gained or lost at each stage. The simulation frees us from the constraints of what is clinically available and lets us ask the deeper question: *where in the nervous system does the diagnostic information actually live?*

To answer this, we add a simulated population of ~80 motor cortex neurons to our reaching dataset. Each neuron fires according to a well-established principle from motor neuroscience: **cosine tuning**.

> Week 5 asked: can EMG tell us what the clinician knows? Week 6 goes one level deeper: can motor cortex neurons tell us even more? We add a neural population to the same reaching task and compare neural decoding with the muscle-based baseline.

2. Cosine Tuning in Motor Cortex

In 1982, Apostolos Georgopoulos and colleagues made a discovery that transformed our understanding of motor cortex. They recorded from individual neurons in the arm area of M1 while monkeys reached in different directions, and found a remarkably simple pattern: each neuron has a **preferred direction (PD)** — the reach direction that makes it fire most vigorously. As the reach direction rotates away from the PD, the firing rate falls off as a cosine:

$$\text{firing rate} = \text{baseline} + \text{depth} \times \cos(\theta - \theta_{PD})$$

The **baseline** is the neuron's minimum firing rate, the **depth** is how strongly it modulates, and $\boldsymbol{\theta_{PD}}$ is its preferred direction. This is called **cosine tuning**, and it is one of the most replicated findings in motor neuroscience.

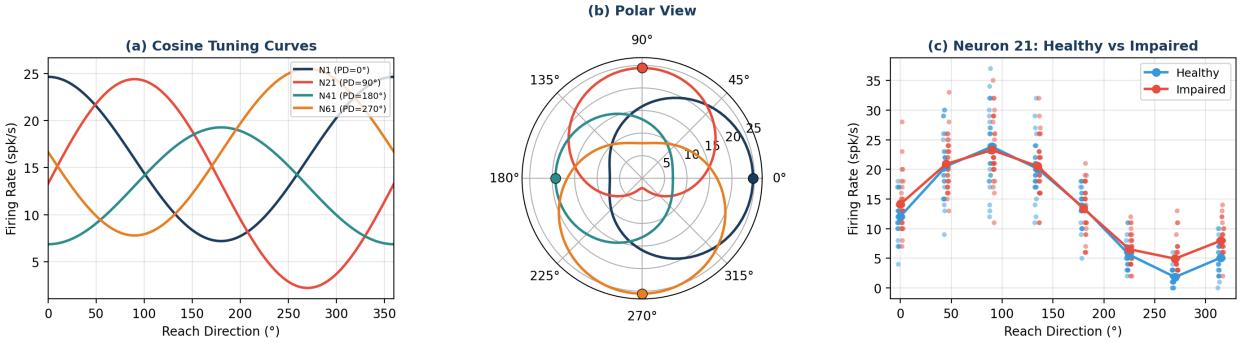

Figure 2. Cosine tuning in motor cortex. (a) Four example neurons with preferred directions at 0°, 90°, 180°, and 270°. Each neuron fires maximally at its PD and minimally at the opposite direction. (b) The same tuning curves in polar coordinates — each neuron's activity traces a circle offset toward its PD. (c) Real data from our simulation: Neuron 21 shows clear cosine tuning in healthy subjects (blue), with reduced modulation depth and increased variability in impaired subjects (red).

No single neuron uniquely specifies the reach direction — its firing rate is ambiguous (the same rate occurs for two different directions). But a **population** of neurons with different PDs collectively resolves this ambiguity. If we sort neurons by their PD and plot their average firing rates for each reach direction, a striking pattern emerges:

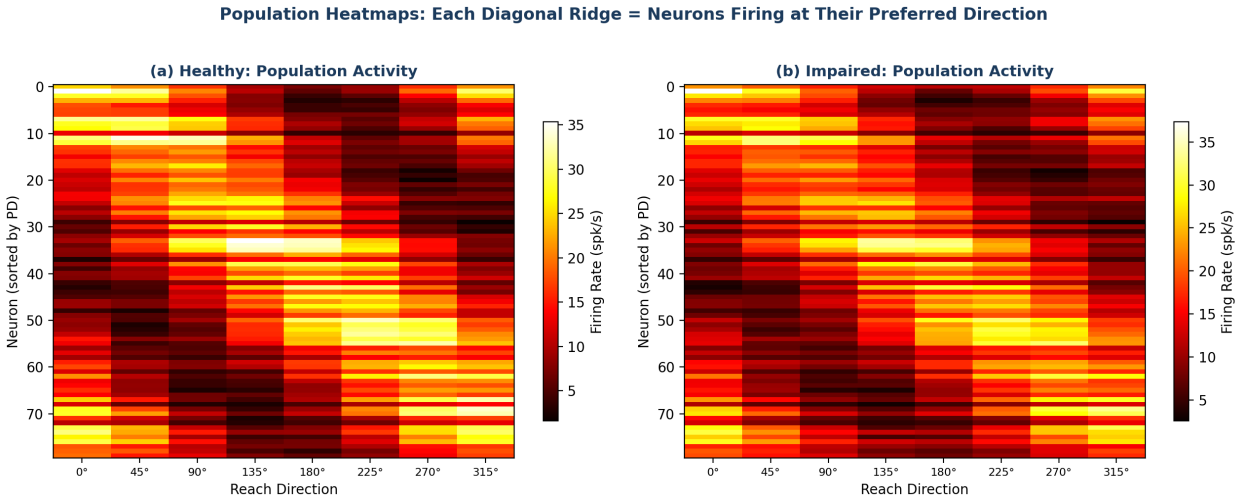

Figure 3. Population activity heatmaps. Neurons are sorted vertically by preferred direction; reach directions are arranged horizontally. (a) Healthy subjects show a sharp diagonal ridge — each direction

maximally activates neurons with matching PDs. (b) In impaired subjects, the ridge is broader and noisier. The synergy merging from Week 4 propagates upstream: if muscles are driven by a reduced set of synergy patterns, the neural commands that produce those patterns also lose their specificity.

But how does the population actually specify a direction? The idea is elegantly simple. Each neuron contributes a vector pointing in its preferred direction, scaled by how vigorously it fires on that trial. Sum all 80 vectors and the result — the **population vector** — points in the direction of the reach:

$$PV = \Sigma_i \, r_i \cdot (\cos \theta_{PD,i}, \sin \theta_{PD,i})$$

where r_i is the firing rate of neuron i and $\theta_{PD,i}$ is its preferred direction. Neurons whose PD is close to the true reach direction fire most strongly, so their votes dominate the sum. Neurons with PDs far from the reach direction contribute little. The result is a vector that points remarkably close to the actual movement direction:

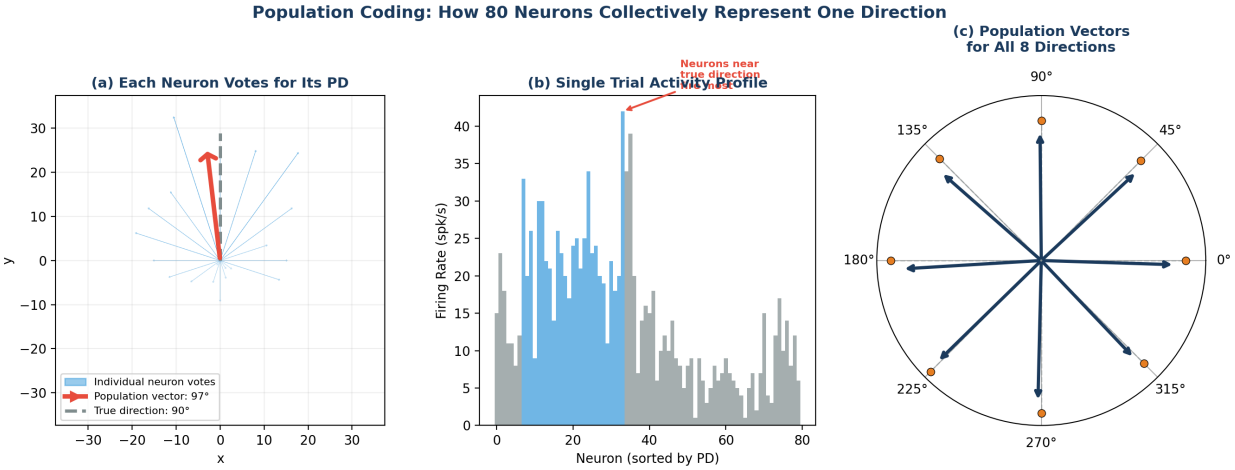

Figure 4. Population coding in action. (a) A single trial reaching at 90°: each blue arrow is one neuron's vote (pointing at its PD, scaled by its firing rate). The red arrow is the population vector — the sum of all votes — pointing at 97°, close to the true direction (dashed). (b) The same trial's activity profile: neurons with PDs near 90° (blue bars) fire most vigorously, creating the peak that drives the population vector. (c) Population vectors for all 8 directions (healthy average), each closely matching the true target (orange dots).

Cosine tuning is the encoding model: it tells us what each neuron does. The population vector is the readout: it sums each neuron's vote to recover the direction of the reach. Impaired subjects show blurred population patterns, consistent with the synergy merging from Week 4. We will formalize the population vector as a decoder in Section 5.

3. Modeling Spike Counts: Why We Need a New Tool

We now want to fit the cosine tuning curves from Section 2 — to estimate each neuron's preferred direction and modulation depth from data. This is a regression problem: given the reach direction on each trial, predict the neuron's firing rate. But if we reach for the tools we already know, we immediately run into trouble.

The problem with linear regression

Spike counts are **non-negative integers**. A neuron might fire 0, 5, 12, or 30 spikes on a given trial — but it cannot fire −3 spikes. Ordinary linear regression knows nothing about this constraint. If the data are noisy (and neural data always are), the best-fit line can predict negative firing rates for directions far from the neuron's preferred direction. This is not an edge case: Figure 5b shows it happening. Negative predictions are biologically meaningless and will distort the estimated tuning curve.

The problem with logistic regression

We learned logistic regression in Week 5, and it served us well for classification. But it was designed for a fundamentally different kind of output: a **probability between 0 and 1**. A neuron's firing rate is not a probability — it might be 5, 15, or 30 spikes per second. The sigmoid function that makes logistic regression so elegant for yes/no questions also caps its output at 1, which makes it useless for modeling count data. We need an output that is always non-negative, can grow as large as the neuron requires, and naturally handles the discrete, skewed distribution of spike counts.

The solution: the exponential link

The answer is to pass the linear predictor through the **exponential function** instead of the sigmoid. The output of exp(·) is always positive and unbounded:

$$\text{firing rate} = \exp(\beta_0 + \beta_1 \cdot \cos(\theta - \theta_{PD}))$$

The linear predictor $\beta_0 + \beta_1 \cdot \cos(\theta - \theta_{PD})$ can be any real number, but the exponential guarantees a positive output. At the preferred direction, cos = 1 and the rate is maximal; at the anti-preferred direction, cos = −1 and the rate is minimal but still positive — exactly the shape we see in cosine tuning curves.

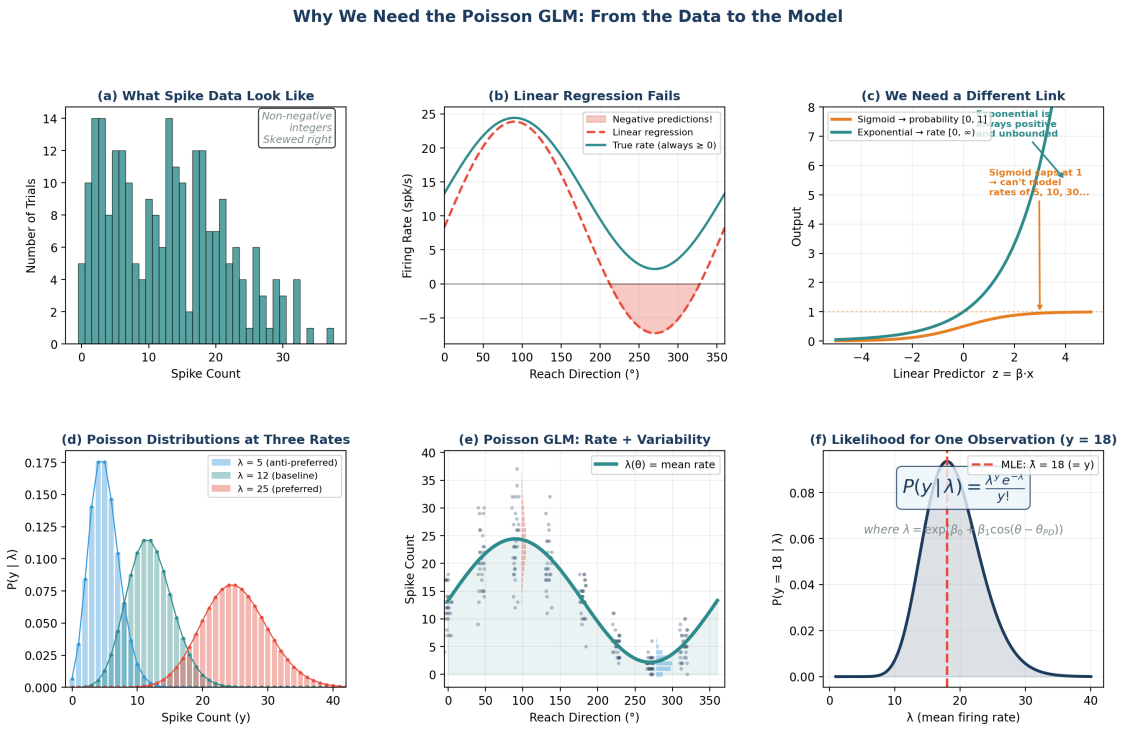

Figure 5. Why we need the Poisson GLM. Top row: (a) Spike counts from one neuron — non-negative integers, skewed right. (b) Linear regression can predict negative rates (red shading). (c) The sigmoid (orange) caps at 1; the exponential (teal) is always positive and unbounded. Bottom row: (d) Poisson distributions at three firing rates — at the preferred direction (λ = 25, wide), the baseline (λ = 12), and the anti-preferred (λ = 5, narrow and skewed). (e) The Poisson GLM: the tuning curve (teal) gives the mean rate, and sideways histograms show the Poisson variability at two directions. (f) Likelihood for observing y = 18 spikes as a function of λ. MLE finds the peak.

The Poisson distribution

Why Poisson, specifically? The Poisson distribution describes the probability of observing exactly y spikes when the underlying mean rate is λ:

$$P(y \mid \lambda) = \lambda^{y} \cdot e^{-\lambda} / y!$$

Two properties make this a natural model for neural spiking. First, the variance equals the mean: when a neuron fires at a high rate, the trial-to-trial variability is also high. This matches what we observe in real cortical recordings. Second, for large λ the Poisson approaches a Gaussian, while for small λ it is naturally skewed rightward — reflecting the fact that you cannot fire fewer than zero spikes (Figure 5d).

Maximum likelihood estimation

Now we have a model (cosine tuning through the exponential link) and a noise distribution (Poisson). But how do we find the best-fit parameters? In Week 5, we mentioned that logistic regression maximizes the *log-likelihood*, without dwelling on what that means. The Poisson GLM gives us a chance to build the intuition.

Imagine you observe a neuron fire y = 18 spikes on a single trial. You do not know the neuron's true mean rate λ. But you can ask: for each possible value of λ, how probable is it that a Poisson process with that rate would produce exactly 18 spikes? This is the **likelihood function** — the

Poisson probability formula, viewed not as a function of y (which we already observed) but as a function of λ (which we want to estimate). Figure 5f plots this curve: it peaks at λ = 18, exactly the observed count. That peak is the **maximum likelihood estimate (MLE)** — the value of λ that makes our data most probable.

For a single trial, the MLE is trivially the observed count. The power of MLE emerges when we have *many* trials at *different* directions. Now λ is not a single number but a function of direction — $\lambda(\theta) = \exp(\beta_0 + \beta_1 \cdot \cos(\theta - \theta_{PD}))$ — and we want to find the parameters (β_0, β_1, θ_{PD}) that jointly maximize the probability of *all* observed spike counts across all trials and all directions. The total likelihood is the product of individual Poisson probabilities (one per trial), and in practice we maximize the sum of their logarithms — the **log-likelihood**:

$$\log L = \Sigma_{trials} [\, y_t \cdot \log \lambda(\theta_t) - \lambda(\theta_t) - \log(y_t!) \,]$$

This is exactly what sklearn's `PoissonRegressor` optimizes under the hood. And the same principle applied in Week 5: logistic regression maximizes a log-likelihood too, just with Bernoulli probabilities instead of Poisson. MLE is the common engine behind all GLMs.

Stepping back: the GLM family

What we have built is a **generalized linear model (GLM)**. The structure is always the same — a linear combination of inputs, passed through a link function — but the link and the noise distribution change to match the data:

- **Linear regression:** identity link, Gaussian noise. For continuous, real-valued responses.
- **Logistic regression** (Week 5): sigmoid link, Bernoulli distribution. For binary outcomes — P(impaired | EMG).
- **Poisson regression** (Week 6): exponential link, Poisson distribution. For non-negative counts — spike rates given reach direction.

The logistic regression we already know and the Poisson regression we need today are the same mathematical machinery with different output assumptions. Recognizing this saves us from treating every new data type as a separate problem — we choose the link function that matches the biology, and MLE does the rest.

> Spike counts are non-negative integers — linear regression can predict negative rates, and logistic regression caps at 1. The Poisson GLM solves both problems: the exponential link guarantees positive outputs, and the Poisson distribution captures the count nature and variance structure of spike data. Together with logistic regression from Week 5, it forms part of the GLM family — same framework, different link functions for different data types.

4. Encoding: What Is Each Neuron Doing?

We have 80 neurons recorded during reaching. We know the direction of each reach. But we do not know what role each individual neuron plays in the movement. Is Neuron 23 involved in rightward reaches? Does Neuron 57 care about upward movements? Which neurons are strongly tuned and which are barely modulated? To build a population decoder (Section 5) or to understand how impairment alters the neural code (Section 6), we first need to characterize each neuron's contribution. This is the **encoding question**: given the movement direction, what should each neuron's activity look like?

Section 2 told us that motor cortex neurons follow cosine tuning. Section 3 gave us the Poisson GLM — a tool designed for exactly this kind of count data. Now we put them together: for each neuron, we fit a Poisson GLM with cosine predictors to estimate its preferred direction and tuning depth from the observed spike counts.

The encoding model

For each neuron, we fit:

$$\text{spike count} \sim \text{Poisson}(\exp(\beta_0 + \beta_1 \cdot \cos(\theta) + \beta_2 \cdot \sin(\theta)))$$

Why $\cos(\theta)$ and $\sin(\theta)$ instead of $\cos(\theta - \theta_{PD})$ directly? Because θ_{PD} is what we are trying to find. If we included it explicitly, we would need nonlinear optimization. The cos/sin decomposition is a standard trick that makes the model linear in its parameters: MLE can find β_1 and β_2 directly, and we recover the preferred direction as $\theta_{PD} = \text{atan2}(\beta_2, \beta_1)$. The modulation depth follows from $\sqrt{(\beta_1^2 + \beta_2^2)}$.

A deliberate choice: we fit using only **healthy subjects**. This gives us the tuning model for a normally functioning motor cortex — the reference against which we can later measure how impairment changes neural coding. Think of it as establishing what each neuron *should* be doing before asking what goes wrong.

How well does the model recover each neuron's identity?

If the Poisson GLM is doing its job, the recovered preferred directions should closely match the true ones. This is a critical validation step — if we cannot accurately characterize each neuron's tuning from data, everything downstream (population decoding, clinical comparisons) is built on a shaky foundation.

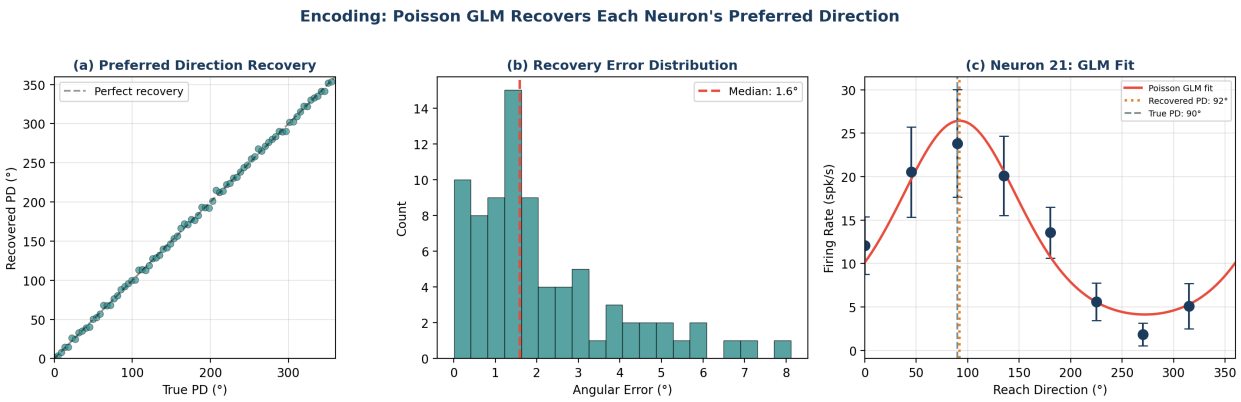

Figure 6. Encoding: recovering each neuron's preferred direction. (a) Recovered vs true PDs for all 80 neurons — points cluster tightly along the diagonal. (b) The distribution of angular errors; median error is just 1.4°. (c) A detailed look at Neuron 21: the Poisson GLM fit (red curve) closely tracks the observed mean firing rates at each direction (blue dots ± SD). The recovered PD (orange) nearly overlaps the true PD (gray).

The recovery is excellent — median angular error under 2° across all 80 neurons. This works so well because cosine tuning is exactly the generative model we used to simulate the data, meaning the Poisson GLM is correctly specified. In real cortical recordings, the fit would be noisier: real neurons have gain modulation, non-cosine tuning components, and history-dependent firing. But the Poisson GLM remains the standard first approximation in systems neuroscience — simple enough to be interpretable, flexible enough to capture the dominant structure.

Notice what we have accomplished: starting with raw spike counts and reach directions, we can now assign each neuron a **preferred direction** (what movement it cares about) and a **tuning depth** (how strongly it cares). These two numbers summarize a neuron's functional role in the reaching circuit. In Section 6, we will ask: do these numbers change in impaired subjects? And if so, which neurons are most affected?

> Encoding asks: given the movement, what is each neuron doing? The Poisson GLM recovers each neuron's preferred direction and tuning depth from spike count data. These two numbers define the neuron's functional identity — its role in the reaching circuit. We fit on healthy data to establish the reference, setting up the clinical comparison in Section 6.

5. Decoding: What Direction Was the Reach?

Section 4 answered the encoding question: given the movement direction, what does each neuron do? Now we turn the question around. A subject reaches, 80 neurons fire, and we observe a vector of 80 spike counts. **What direction was the reach?** This is the **decoding question** — the same question we asked of muscle data in Week 5, now applied one level upstream in the motor pathway.

Decoding matters for two reasons. Practically, it is the foundation of brain-computer interfaces: if a paralyzed patient cannot move their arm, a decoder that reads the intended direction from motor cortex can drive a robotic limb. Scientifically, decoding accuracy tells us how much information the population carries about the movement — and by comparing neural decoding to muscle decoding, we learn which level of the motor pathway is more informative for a given task.

Why different models for encoding and decoding?

A natural question: we just used the Poisson GLM for encoding — why not use it for decoding too? The answer lies in what each model predicts. The Poisson GLM models **spike counts given direction**: P(spikes | θ). Its output is a firing rate, not a direction label. To decode, we need the reverse: **direction given spike counts**. Logistic regression does exactly this — it takes a vector of features (80 firing rates) and outputs the probability of each direction class. The Poisson GLM and logistic regression are complementary: one goes from movement to neural activity, the other goes from neural activity back to movement.

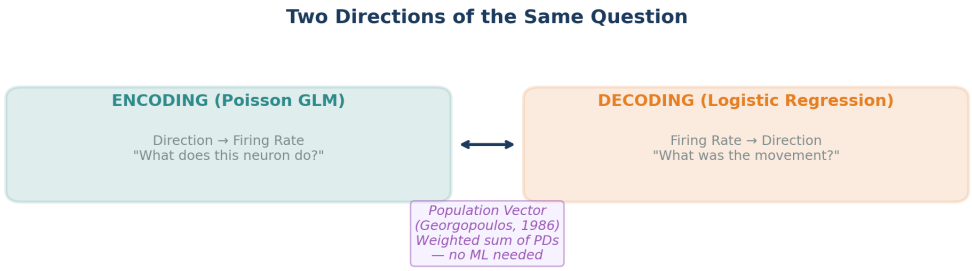

Figure 7. Two directions of the same question. Encoding (left): predict neural activity from direction — the Poisson GLM. Decoding (right): predict direction from neural activity — logistic regression or the population vector. The Poisson GLM cannot decode because it outputs spike rates, not direction labels; logistic regression cannot encode because it outputs class probabilities, not firing rates.

The population vector — a decoder that needs no training

Before using logistic regression, there is a simpler approach that requires no training at all. In Section 2, we saw that the population vector — the weighted sum of each neuron's PD vector — points in the direction of the reach. Why does this work *without* any machine learning?

The key insight is that the encoding model already contains everything the decoder needs. If we know each neuron's preferred direction (from Section 4), the decoding rule follows directly from

the cosine tuning assumption: neurons whose PD is close to the true direction fire most, so their votes dominate the sum. We are not learning a mapping from data — we are *inverting* the encoding model analytically. No training set, no test set, no cross-validation, no hyperparameters. Just the PD estimates from Section 4 and the firing rates from a single trial:

$$PV = \Sigma_i \, r_i \cdot (\cos \theta_{PD,i}, \sin \theta_{PD,i}) \rightarrow \text{decoded direction} = \text{angle}(PV)$$

To evaluate accuracy, we compute the population vector for every trial, find the nearest of the 8 target directions, and check whether it matches the true label:

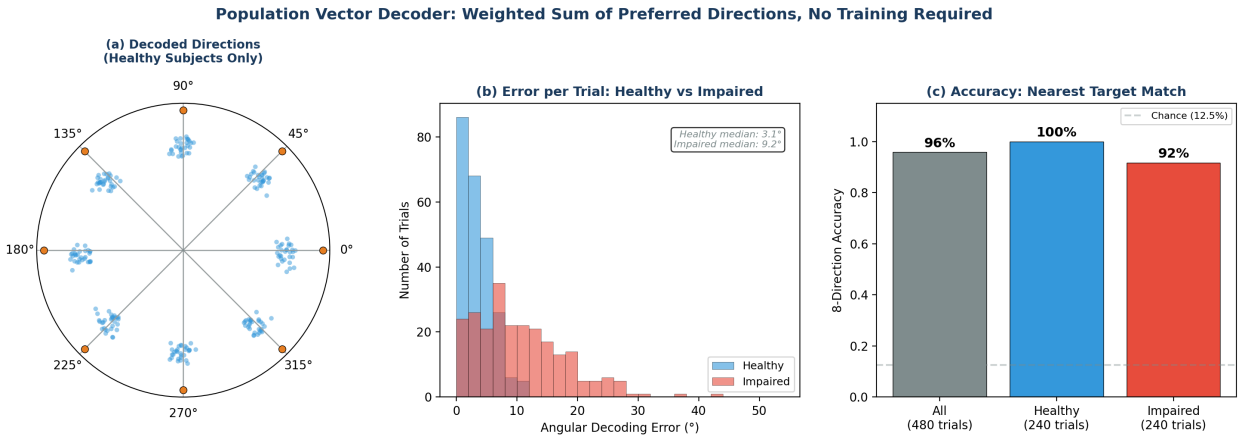

Figure 8. Population vector decoder. (a) Decoded directions for healthy subjects only — each dot is one trial, clustered tightly around the true target directions (orange markers). (b) Angular error per trial: healthy subjects (blue) have a median error of 3°; impaired subjects (red) show wider errors (median 9°) because synergy merging blurs their neural tuning. (c) When we assign each trial to the nearest target direction, overall accuracy is 96%. Healthy subjects achieve 100%; impaired subjects drop to 92% — still strong, but the gap reveals that their population code is degraded.

96% accuracy from a decoder with zero tunable parameters is striking. It tells us that the cosine tuning model is a powerful description of these neurons — so powerful that simply inverting it analytically suffices for decoding. The population vector is historically important (Georgopoulos introduced it in 1986 and it remains the standard first-pass decoder in motor neuroscience), conceptually clean, and surprisingly competitive. But it has two limitations: it cannot adapt to correlations between neurons, and it can only decode direction — it cannot classify healthy vs impaired.

Logistic regression on neural features

For both direction decoding and clinical diagnosis, we need a decoder that learns from data. We use the exact same logistic regression pipeline from Week 5 (C = 10, LOSO cross-validation), but replace the 6 muscle amplitudes with 80 neural firing rates. Everything else stays identical — same algorithm, same evaluation, same clinician labels. This isolates the effect of the **representation**:

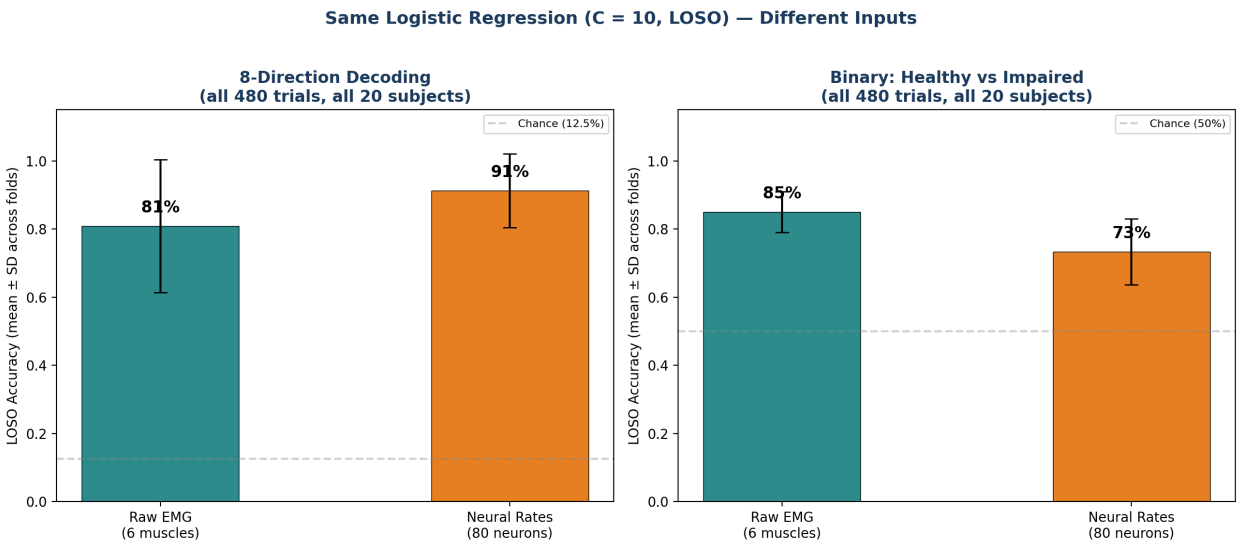

Figure 9. Logistic regression: same algorithm, different inputs. Each bar is the mean LOSO accuracy across 20 subject folds; error bars show ± 1 SD across folds. Left: 8-direction decoding using all 480 trials from all 20 subjects — neural features (91%) outperform EMG (81%). Right: binary healthy-vs-impaired classification on the same 480 trials — EMG (85%) outperforms neural features (73%).

For **direction decoding**, neural features win decisively: 91% vs 81%. This makes sense — motor cortex neurons encode direction explicitly (it is their primary job), so 80 cosine-tuned channels carry a cleaner directional signal than 6 muscles that mix direction with force magnitude, synergy structure, and biomechanics.

For **clinical diagnosis**, muscles win: 85% vs 73%. This is the more revealing result, and we explore it in depth in Section 6.

> Decoding goes from neural activity to movement. The population vector (96%) inverts the encoding model analytically — no training needed. Logistic regression on neural features (91%) outperforms muscle-based decoding (81%) for direction, but muscles win for clinical diagnosis (85% vs 73%). Same algorithm, different data — the representation determines the answer.

6. The Clinical Question: Where Does the Diagnostic Signal Live?

Section 5 revealed a striking asymmetry: neural features decode *direction* better than muscles (91% vs 81%), but muscles diagnose *impairment* better than neurons (85% vs 73%). Why would more data from an earlier stage of the motor pathway produce worse diagnostic performance? The answer lies in where the impairment actually acts.

Tracing the impairment through the motor pathway

Recall from Week 4 that we modeled impairment as **synergy merging**: healthy subjects coordinate their muscles using three independent synergy patterns, while impaired subjects merge two of those synergies into one. This merging happens at the spinal level — below motor cortex, above the muscles. The consequence is that the impairment affects the two ends of the motor pathway very differently.

Start at a single neuron. Panel (a) of Figure 10 shows Neuron 21's tuning curve for healthy (blue) and impaired (red) subjects. The difference is subtle: the impaired curve has a slightly lower peak and slightly higher trough. The error bars (standard error of the mean across trials) nearly overlap at most directions. If you saw this neuron in isolation, you might not notice anything wrong.

Now zoom out to the full population. Panel (b) plots the modulation depth (peak rate minus trough rate, averaged across directions) for all 80 neurons: healthy depth on the x-axis, impaired depth on the y-axis. If impairment had no effect, every point would sit on the diagonal. Instead, 72 of 80 neurons fall below it — their tuning is systematically shallower in impaired subjects. The effect is consistent but small: most points hover near the diagonal, not far below it.

Now look at the muscles. Panel (c) shows the mean EMG amplitude for each of the 6 muscles, with healthy and impaired subjects side by side. The differences are immediately visible — certain muscles show large amplitude changes between groups. This is because synergy merging directly reshapes the 6-dimensional muscle activation pattern. The muscles sit downstream of the impairment and feel its full force.

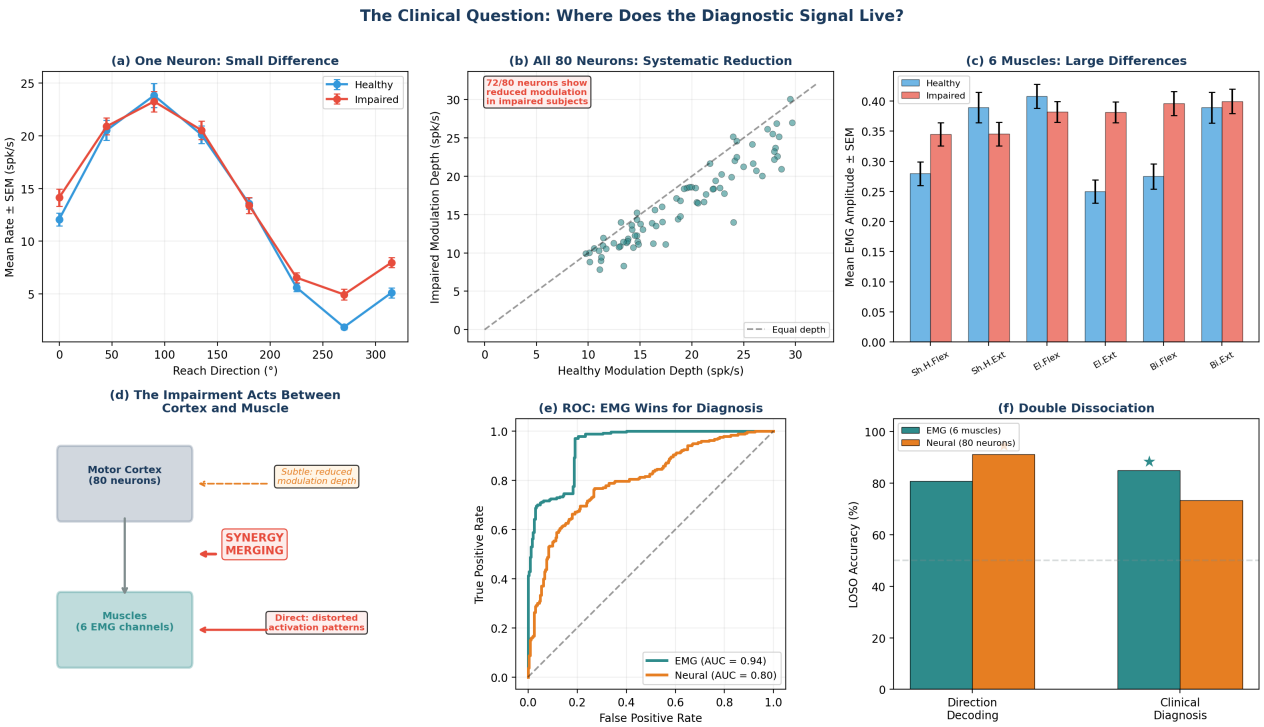

Figure 10. Where does the diagnostic signal live? Top row: the impairment at two levels. (a) A single neuron shows a subtle tuning difference between healthy and impaired — error bars nearly overlap. (b) Across all 80 neurons, modulation depth is systematically but modestly reduced (72/80 below diagonal). (c) The same impairment produces large, visible differences in muscle amplitudes. Bottom row: the diagnostic consequence. (d) Synergy merging acts between cortex and muscle, so muscles feel the direct effect while cortex shows only secondary changes. (e) ROC curves confirm: EMG (AUC = 0.94) outperforms neural features (AUC = 0.80) for diagnosis. (f) The double dissociation: neurons win for direction decoding, muscles win for clinical diagnosis.

Why the asymmetry?

Panel (d) makes the mechanism explicit. Synergy merging acts at the spinal level, *between* motor cortex and the muscles. The motor cortex still generates relatively normal directional commands — it is trying to produce the right movement. But those commands pass through a corrupted synergy transformation on the way down to the muscles. The result: neurons carry a strong directional signal (that is their primary job, and the impairment does not directly disrupt it), but only a weak diagnostic signal (the impairment affects them indirectly, through subtle changes in feedback and coordination). Muscles carry a weaker directional signal (direction is mixed with force, biomechanics, and synergy structure), but a strong diagnostic signal (the impairment directly distorts their activation patterns).

The ROC curves in panel (e) quantify this: EMG achieves an AUC of 0.94 for the healthy-vs-impaired classification, while neural features reach only 0.80. Panel (f) summarizes the **double dissociation**: the best recording site depends on the question. Neurons for direction, muscles for diagnosis.

The broader lesson

This result extends beyond our simulation. In real clinical practice, the choice of recording site is often driven by convenience or tradition — EMG because it is non-invasive, cortical recordings

because they are ‘closer to the source.’ But our analysis shows that proximity to the source is not always an advantage. The best measurement depends on what you want to know, and where in the system the relevant information lives. Machine learning does not change this — no algorithm can extract a signal that the data do not contain.

> The diagnostic signal lives where the impairment acts. Synergy merging distorts muscle patterns directly (AUC = 0.94) but reaches motor cortex only indirectly (AUC = 0.80). This produces a double dissociation: neurons decode direction better, muscles diagnose impairment better. No algorithm can compensate for recording from the wrong level of the motor pathway.

7. Bringing It Together

This week we traveled upstream in the motor pathway — from the muscles we have been classifying since Week 4 to the motor cortex neurons that drive them. We learned how those neurons encode direction (cosine tuning), built a model to fit their responses (the Poisson GLM), decoded movement from their activity (population vector and logistic regression), and discovered that the best recording site depends on the question we are asking. Where does all of this fit in the bigger picture?

The updated comparison table

Week 5 established the first row of our running comparison table: logistic regression on raw EMG (80.8% direction, 85.0% binary, AUC 0.94). This week, we kept the algorithm identical and changed the data. The table now has three rows:

Running Comparison Table — Updated Through Week 6

Week	Method	Features	8-Dir Accuracy	Binary Accuracy	AUC
5	Logistic Reg (C = 10)	Raw EMG (6 muscles)	80.8%	**85.0%**	**0.94**
6	Logistic Reg (C = 10)	Neural Rates (80 neurons)	**91.2%**	73.3%	0.80
6	Population Vector	Neural Rates (80 neurons)	**95.8%**	N/A (direction only)	N/A
7	*Naive Bayes*	*Both*	*?*	*?*	*?*
8	*KNN*	*Both*	*?*	*?*	*?*

EMG columns (Week 5) hold steady. Neural columns are new this week. Weeks 7–11 add new algorithm rows evaluated on both feature sets.

Figure 11. Running comparison table through Week 6. Row 1 (Week 5 baseline): logistic regression on EMG. Row 2: the same logistic regression on neural rates — direction accuracy jumps from 81% to 91%, but binary diagnosis drops from 85% to 73%. Row 3: the population vector, which decodes direction at 96% with no training at all — but it can only decode direction, not classify healthy vs impaired (hence N/A for binary and AUC). Weeks 7–11 will add new algorithm rows, each evaluated on both feature sets.

Three lessons from the table

First, **the representation changed the answer more than any algorithm could.** Swapping 6 EMG channels for 80 neural firing rates — with the same logistic regression — moved direction accuracy from 81% to 91%. No amount of hyperparameter tuning in Week 5 (C, regularization type, feature scaling) produced a gain that large. The data you feed the model matters at least as much as the model itself.

Second, **a model-free decoder can outperform a trained classifier.** The population vector (96%) beats logistic regression on the same neural data (91%) for direction decoding. It has zero tunable parameters — it simply inverts the cosine tuning model analytically. When the encoding model is well-matched to the data, the best decoder may not be a machine learning algorithm at all.

Third, **the best method depends on the task.** For direction decoding, neural features with the population vector win (96%). For clinical diagnosis, raw EMG with logistic regression wins

(85%, AUC 0.94). There is no single best row in this table — and there may never be. The double dissociation from Section 6 means that the optimal choice of features and method depends on what question you are asking.

What comes next

Starting in Week 7, we add new algorithms: Naive Bayes, KNN, SVM, Random Forests, and LDA. Each will fill in a new row, evaluated on *both* EMG and neural features. The question shifts from 'does richer data help?' (answered this week: it depends on the task) to 'does a smarter algorithm help?' Can a nonlinear classifier like SVM extract diagnostic information from neural rates that logistic regression misses? Can an ensemble method like Random Forests handle the 80-dimensional neural feature space better than a linear model? By Week 14, the table will be full, and we will be able to ask the capstone question: across all methods and all representations, what gets closest to the clinician?

What we learned this week

- **Motor cortex neurons** encode reach direction via cosine tuning (Georgopoulos, 1982). Each neuron has a preferred direction; the population collectively specifies the movement through a weighted sum of preferred-direction vectors.
- **Spike counts require a new model.** Linear regression predicts negative rates; logistic regression caps at 1. The Poisson GLM (exponential link, Poisson noise) is designed for count data and fits tuning curves via maximum likelihood estimation.
- **Encoding vs decoding** are complementary. The Poisson GLM encodes (direction → firing rate); logistic regression and the population vector decode (firing rate → direction). Different link functions, different directions of prediction.
- **The population vector** — a parameter-free decoder that inverts the cosine tuning model analytically — achieves 96% direction accuracy, outperforming logistic regression (91%) on the same data.
- **A double dissociation** emerges: neural features decode direction better (91% vs 81%), but muscle features diagnose impairment better (85% vs 73%, AUC 0.94 vs 0.80). The diagnostic signal lives where the impairment acts.
- **Representation can matter more than algorithm.** Changing the input data (EMG → neural) moved direction accuracy by 10 percentage points. No hyperparameter change in Week 5 came close.

Before reaching for a fancier algorithm, ask whether you are measuring the right thing. This week showed that the choice of representation — muscles vs neurons — changed performance more than the choice of model. And the right representation depends on the question: neurons for direction, muscles for diagnosis. Starting next week, we vary algorithms while holding both feature sets fixed, and the comparison table begins to fill.